

SmartBall: Sensor-equipped Ball for Sports Analytics

Anthony Lozbin, Pritkumar Pagda, Jacob Moran – CIS4930-005 Wireless & Mobile Computing

Abstract

Traditional sports analytics systems rely on expensive, complex, and typically immobile infrastructure such as radar which significantly reduces accessibility and practicality which leaves smaller sports organizations without access to important analytical data. This project introduces the SmartBall, a low-cost, portable alternative that directly embeds wireless sensing and communication capabilities directly into a game ball. The system uses an ESP32 microcontroller and an MPU6050 accelerometer-gyroscope to capture motion data during a throw. When the SmartBall detects a throw event based on acceleration and rotational thresholds, the SmartBall transmits sensor data via UDP over Wi-Fi to a base station. A Python-based listener processes the received data in real-time to compute performance metrics including release time, peak acceleration, peak rotational velocity, and estimated throw distance. Experimental evaluation conducted in a controlled environment with 38 valid trials demonstrated an average estimated throw distance error of approximately 4% which indicates consistency and accuracy for a velocity-based prediction algorithm. While the SmartBall prototype has limitations such as reliance on a controlled environment, increased ball size and weight, and operating only in the sensor frame, the results show the feasibility of embedded, sensor-based sports analytics. Future development of the SmartBall includes global frame tracking, reducing form factor and weight, and improved communication robustness, all of which could further improve the system's practicality and performance.

1. Introduction

Sports analytical data has become increasingly important in both professional and amateur level sports which is driven by a continuous demand for increasing the performance of athletes (Mănescu, 2025). In many professional sports organizations, the analytical systems in place often rely on radar tracking, high-speed cameras, and complex sensor networks across a playing field. While these systems are effective, they impose a high financial burden, complex setup requirements, and limit portability. These drawbacks make advanced sports analytics largely inaccessible outside of professional or well-funded sports organizations.

To address these constraints, we developed the SmartBall, which is a self-contained, sensor-equipped game ball that can capture and transmit motion data wirelessly in real-time. This is done by embedding sensor and communication capabilities directly into a game ball, eliminating the need for external tracking infrastructure like radar. Additionally, the SmartBall's wireless communication capabilities allow it to operate in a wide range of environments, significantly improving portability and practicality while reducing the cost burden immensely by using cheap commercial-off-the-shelf components (COTS).

The SmartBall system is built and operates around two main components: the hardware-embedded game ball and a base station used for receiving and processing communicated data. The hardware-embedded game ball

uses an ESP32 microcontroller and an MPU6050 accelerometer-gyroscope inertial measurement unit (IMU). When the game ball detects a throw based on pre-defined motion thresholds, the system transmits the motion data captured by the MPU6050 over Wi-Fi using UDP to the base station. A Python-listener running on the base station processes the transmitted sensor data to extract sports performance metrics such as release time, peak acceleration, peak rotational velocity, and estimated throw distance.

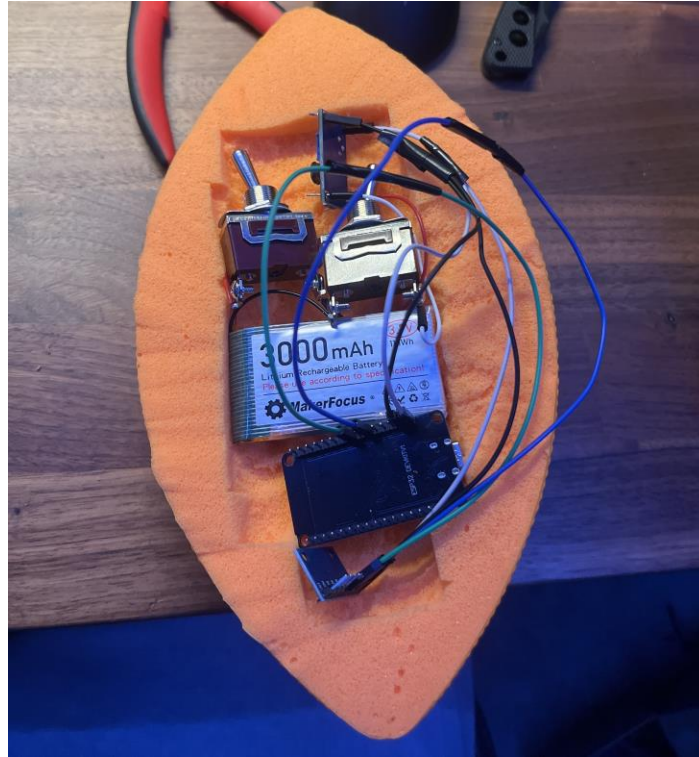


Figure 1: SmartBall Prototype during internal hardware component fitment testing.

The figure above shows the SmartBall prototype during an early component fitment test. At this stage, the internal hardware components were placed inside the prototype game-ball to test whether the ESP32, MPU6050 sensor, and supporting components could physically fit within the available space. While this does not show the entire system architecture, it shows an overview of the prototype's embedded design and demonstrates our core idea of integrating sensing, communication, and power capabilities directly into a game ball. This physical integration of these components is what allows the SmartBall to collect motion data and transmit said motion data, while being a self-contained system. Thus our contributions are as follows: (1) the design and implementation of a low-cost, portable sports analytics system, (2) a real-time data processing pipeline which is capable of extracting sports performance metrics from raw IMU data, and (3) an experimental evaluation demonstrating accurate distance estimation with minimal error in a controlled environment.

2. System Design

The SmartBall system is designed as an embedded, wireless sensing system that can capture, transmit, and analyze motion data in real-time. The system is composed of two main components: the hardware-embedded game ball and a base station which is responsible for receiving and processing the motion data that the game

ball transmits. The embedded system monitors motion continuously to detect a throw and only transmits the motion data when a throw is detected to reduce the overhead associated with data transmission. The base station, which is any computer or device equipped with Wi-Fi and capable of running a Python program, receives transmitted data and performs real-time computations to extract the sports performance metrics.

One of our key design goals for the SmartBall system is to minimize reliance on external infrastructure. Unlike traditional sports analytics systems that depend on fixed radar or camera setups, the SmartBall integrates sensing and communication directly into the ball. This immensely reduces the portability issues associated with traditional systems and allows the system to operate in a variety of environments.

The SmartBall system operates in three stages. First, the hardware-embedded game ball collects motion data from the MPU6050 IMU. Next, the ESP32 microcontroller processes this data to detect a throw event based on pre-defined motion thresholds and when a throw event occurs, transmits relevant samples over Wi-Fi using UDP. Lastly, a Python listener running on the base station receives data and performs analysis in real-time to obtain the various sports performance metrics. The results are both displayed in real-time and logged for future analysis. The following sections describe the system design in depth which includes the hardware components, embedded software implementation on the ESP32 microcontroller, and the data processing pipeline implemented in the listener.

2.1 Hardware Overview

The SmartBall hardware system is centered around an embedded hardware architecture that integrates sensing, processing, power management, and wireless communication within a compact enough form factor to fit within a game ball. The primary components include an ESP-32S microcontroller, MPU6050 6-axis accelerometer-gyroscope IMU, MT3608 DC-DC boost converter, 3.7V 3000mAh LiPo battery, and mechanical toggle switches to manage power delivery across the system. The ESP32 operates as the core processing and communication unit within the system. It is responsible for interfacing with the MPU6050 to receive motion data, perform data handling, detect when a throw occurs, and transmit sensor data via UDP over Wi-Fi. The MPU6050 provides acceleration and angular velocity measurements across three axes, enabling our system to capture both linear and rotational motion of the game ball during a throw.

All non-mechanical components of the system require power to operate which is supplied by a 3.7V 3000mAh LiPo battery. The ESP32 and MPU6050 used within the system both require 5V as input to operate correctly and thus we rely on the MT3608 DC-DC boost converter to step up the 3.7V output of the LiPo battery to the 5V required. The system includes switching mechanisms on the battery terminals to allow for safe power control and easier battery removal from the system. The ease of battery removal is important as charging the battery is done outside of the system using a TP4056 USB-C 5V 1A battery charging module, which requires the battery to be disconnected from the circuit. In Figure 2 seen below, two switches can be seen. The reason for the negative terminal of our LiPo battery being switched is to prevent the battery from having to be de-soldered and re-soldered when charging is required as the terminals are connected to the switches via a screw-contact.

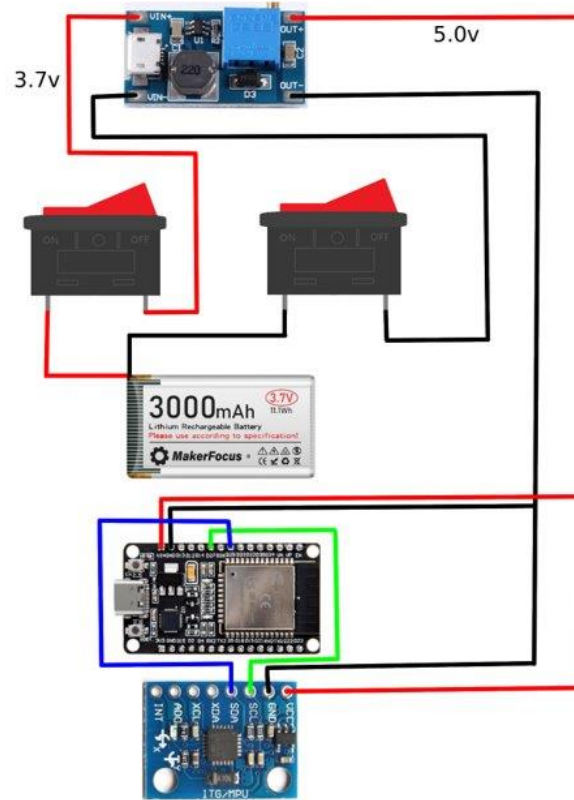


Figure 2: Hardware Architecture of the SmartBall

The figure above (Figure 2) shows the hardware configuration of the SmartBall system, including the connections between the various components of the system. This figure shows the process through which the 3.7V power source is converted into the necessary 5V as well as the connection between the ESP32 and MPU6050. After the 3.7V is boosted to 5V, this 5V connection is tied to the VCC pins of both the ESP32 and MPU6050, while the negative terminal is connected to the GND pins of both the ESP32 and MPU6050. For transferring data from the sensor to the ESP32, the SDA and SCL pins of the MPU6050 are connected to the D25 and D27 pins of the ESP32 respectively. The design reflects the balance between functionality and physical constraints, as all components must fit within the limited internal space of a game ball while still maintaining reliability. This design enables the SmartBall to function as a self-contained sensing device without the need for any external dependencies outside of the base station running the listener.

2.2 ESP32 Code

The ESP32 program of the SmartBall system acquires inertial sensor data (MPU6050) and informs about identified throws via UDP protocol. This part initializes the I2C communication with SDA_PIN = 25 and SCL_PIN = 27 pins for ESP32 board. The MPU6050 sensor is configured to operate at high-g mode. Additionally, connection to Wi-Fi is established, and data from accelerometer and gyroscope sensors are transmitted to the UDP receiver.

The main loop works as a sampling system with a fixed sampling rate. The `micros()` function together with the constant `SAMPLE_US = 5000` ensures the frequency of 200 Hz, which corresponds to one sample per each 5 milliseconds. Each sample consists of acceleration values a_x, a_y, a_z in m/s^2 and gyroscope values g_x, g_y, g_z in rad/s obtained from the inertial sensor. Before operating normally, gyroscope calibration is needed; for this purpose, 500 samples were taken in the stationary position and averaged using `calibrateGyro()`.

A moving circular buffer is utilized for the pre-roll containing 50 samples of 250 milliseconds when the sampling frequency is set at 200Hz. With this algorithm, we will be able to capture the movement just before starting the throwing action. The condition for recording is that either the acceleration reaches more than the threshold `ACC_TRIGGER = 18.0  $\text{m/s}^2$` or the angular speed exceeds the threshold value `GYRO_TRIGGER = 6.0  $\text{rad/s}$` , in which case the algorithm will start the recording process, transmitting the START message, followed by the pre-roll data, the trigger sample, and live samples for 900ms.

UDP is sent through the `WiFiUDP` object. The sample from each sensor consists of a packet with timestamp, acceleration components, and gyroscope components:

`time_ms ,ax, ay, az, gx, gy, gz`

The recording state ensures that continuous retransmissions are not possible due to the repeated crossings of the threshold. Once 900 milliseconds worth of throwing data is collected by the ESP32, it will transmit an END packet and exit the recording state, after which it will cool down for at least 1500 milliseconds before it can accept another throw event.

2.3 Listener Code

The listener component of the SmartBall system consists of a script that collects real-time data from the embedded system inside the ball, which includes the timestamp, accelerometer readings, and gyroscope readings. The pipeline in the UDP listener script listens to incoming UDP packets to collect inertial measurement unit data while the ball is thrown, calculates the ball's release point, determines the ball's release speed, identifies the strength of the throw, predicts the distance the ball will travel, and logs the collected information into a CSV file. The script runs on an event-driven protocol, where the sender sends a "START" command to the receiver, processes the throw, sends the timestamped accelerometer and gyroscope data while the throw is being processed, and finally sends an "END" command when the throw ends.

In the beginning, a UDP socket is created which listens to the external ESP32 transmitting UDP packets.. The listener will be able to receive packets of information from any interfaces linked to the listening machine. It is worth mentioning that the listener can be run using microcontrollers or embedded devices capable of communicating via local networks and sending IMU data packets. After setting up, the program goes into a continuous loop and receives packets using `recvfrom(1024)`. Incoming packets are read as a text file, and leading spaces are stripped.

The communication protocol is deliberately sparse. When the command "START" is detected, the script initializes recording to be true, empties the sample buffer, and starts collecting sensor rows. When the command "END" is detected, the script halts recording and submits the collected samples to `process_throw()`. Other commands sent while recording is active are considered to be sensor samples separated by commas. The format is:

`t_ms, ax, ay, az, gx, gy, gz`

First comes the timestamp in milliseconds. This is followed by three more values, each one corresponding to the acceleration on the x, y, and z axes, respectively. The last three values are for the angular velocity on the x, y, and z axes, using data from the gyroscope sensor. The Python program tries its best to convert all the seven fields into floats and append the row only if it has all the seven numerical values.

Prior to any analysis being performed, the script sets a minimum threshold of samples with `MIN_SAMPLES = 25`. By doing so, the program does not attempt to analyze the data if there were no sufficient samples collected. After enough data is collected, the samples are transformed into a NumPy array in order to perform calculations. The timestamp index is set to zero for the first sample, and then the sampling time interval is calculated via the median of the difference between the timestamps. Calculating the timesteps using the median ensures more stability than when calculating the mean.

The algorithm then separates the acceleration and gyroscope data into three-dimensional vectors. It computes the magnitude of acceleration using the Euclidean norm:

$$\text{acc_mag} = \sqrt{ax^2 + ay^2 + az^2}$$

and then similarly computes the gyroscope magnitude:

$$\text{gyr_mag} = \sqrt{gx^2 + gy^2 + gz^2}$$

The above measurements are used for peak measurement, pre-roll validation, and release detection. The peak acceleration measurement and peak angular velocity are saved in the diagnostic variable and then eventually outputted to the CSV file. Both values can be used to gauge the strength of the throw, as well as whether the trajectory seems realistic.

One of the most vital components in the algorithm is the pre-roll baseline estimation procedure. The script is programmed to assume that the start of the recorded data is the pre-roll period when the ball has not yet been thrown. The length of the pre-roll period is specified via `PRE_ROLL_SEC = 0.15`, with a minimum of eight samples. Over the duration of this period, the average acceleration vector is considered the baseline for gravity/body frame. The baseline will be subtracted from the acceleration vector:

$$\text{lin_acc} = \text{acc} - \text{g_body}$$

It is a straightforward approach to compensating for the effects of gravity. It takes the assumption that the device orientation is constant enough throughout the pre-roll stage, which means that the measured acceleration can be considered the gravity vector, and then subtracted from the raw readings to produce a pure linear acceleration of the device. It is far from being an optimal solution compared to using an orientation-aware inertial navigation system, but it is perfectly sufficient for our experimental purposes because it does not require any quaternion-based calculations or magnetometer corrections at all.

As for the pre-roll stage, the script also determines whether it was completely stationary. The decision is made on the basis of the average magnitude of gyroscope data within the range of pre-roll samples. The maximum value of `WARN_PRE_GYRO_MEAN`, which triggers the printing of a warning message, equals 2.5. In case if it is exceeded, the script will still perform further processing and set the status parameter to `ok_warn_preroll`.

The detection of release is done using the `detect_release()` method. The underlying logic behind this method is the fact that when the object is flying freely, the magnitude of the overall acceleration detected must be decreasing to very small values due to the absence of external force exerted on the sensor by hand. The magnitude of the acceleration is averaged using a sliding window approach involving five samples and the first time it falls below free-flight limit is searched for by:

$$\text{FREE_FLIGHT_THRESH} = 0.65 * G$$

where $G = 9.81 \text{ m/s}^2$. Thus, the threshold is equal to approximately 6.38 m/s^2 . The duration of the drop must be greater than $\text{FREE_FLIGHT_MIN_SEC} = 0.020$ seconds, translated to a number of samples based on the median value of the sampling period. Sustained conditions detection minimizes the probability that a single erroneous measurement may be identified as the start point.

In case of a failed sustained free-flight condition search, the routine resorts to a derivatives technique. That is, the routine calculates the derivative between neighboring values of the smoothed magnitude of the acceleration vector, and takes the maximum negative drop as the index to consider. Fallback to such a technique is motivated by the fact that in most cases, release coincides with a significant drop in measured acceleration magnitude.

Once the release index is identified, a few sanity tests are ran on the script. First, the release will be rejected if its index is too close to either side of the sample buffer because this means that there would not be enough sample points preceding or succeeding the release point for the calculation to be accurate. The release will also be rejected if it happens at or past $\text{MAX_RELEASE_FRACTION} = 0.90$ into the capture window since it may have happened too early for the script to receive the `END` command and could be a result of the script's mistake. Finally, the release time must also fall between $\text{MIN_RELEASE_TIME_SEC} = 0.12$ and $\text{MAX_RELEASE_TIME_SEC} = 0.95$.

When a sanity test fails for any particular release, the algorithm will record a CSV line with diagnostic information as well as the type of fail (`bad_release_index`, `release_too_late`, or `release_time_out_of_range`). It is helpful that failed releases are not ignored silently, but rather can be used later for debugging and calibration.

For successful release detections, release velocity is estimated using numerical integration of the linear acceleration prior to release. To minimize the error due to drift, full integration from the beginning of the throw is avoided. Instead, the script will integrate over the last $\text{MAX_INTEGRATION_WINDOW_SEC} = 0.30$ seconds prior to the release point, starting at the later of pre-roll end or `release_idx - max_window_n`.

Initialization of velocity:

$$\mathbf{v} = [0, 0, 0]$$

The script then integrates acceleration using the trapezoidal rule:

$$\mathbf{v} += 0.5 * (\text{lin_acc}[i - 1] + \text{lin_acc}[i]) * \text{dti}$$

This approximates a velocity vector by adding up acceleration vectors in small time intervals. This is more precise than a rectangle sum since it involves averaging two adjacent acceleration vectors within each time interval. After computing the velocity vector, we determine the release speed from the velocity vector magnitude:

$$\text{speed} = ||v||$$

It is then verified whether the calculated speed falls within appropriate speed limits. Any speed that is less than 0.5 m/s, or more than 10.0 m/s, is deemed unrealistic for this demonstration and is classified under the category of `invalid_speed`.

In the case that the throw is deemed acceptable, the program uses a linearly calibrated quadratic equation to calculate the expected distance of the throw:

$$\text{expected_distance_m} = \text{DIST_COEFF} * \text{speed}^2 + \text{DIST_BIAS}$$

With the current setup, `DIST_COEFF` = 0.055 and `DIST_BIAS` = 0.0. However, this is not the full projectile motion physics, but is a reduced model that relates the throwing speed to the expected distance covered. The physical modeling of projectiles requires the initial throwing angle, initial height above ground level, air resistance considerations, and spin among others. Here we use an empirically determined coefficient which is sufficient for generating consistent results for the demonstration purpose.

The throw is also classified into a strength label using `classify_throw()`. The classification thresholds are speed-based, with `speed` < 2.0 being weak, between 2.0 and 4.5 being medium, and `speed` > 4.5 being strong. This provides a simple categorical result that can be displayed to a user or used in a training interface.

Finally, the script writes the processed throw to the throw log file. Each processed throw logs the timestamp, sample count, median timestep, release index, release time, release speed, peak acceleration, peak gyroscope magnitude, expected distance, strength label, and processing status.

As a whole, this listener pipeline presents a streamlined and robust IMU throw analysis system. It receives live input of UDP sensor data, splits throws by START/END indicators, applies basic filtering of signal and throw detection, deducts an average acceleration of pre-throw phase as a baseline to derive linear acceleration, estimates the release velocity via integration of acceleration, excludes outliers, transforms velocities into distances and strengths, and logs all results.

3. Results

The SmartBall system was evaluated in a controlled environment for our experiment to assess its ability to accurately estimate throw characteristics with focus on estimating travel distance. The system successfully detected and processed multiple throw events as well as successfully extracting the sports performance metrics from the transmitted data. Across all valid trials, the system demonstrated consistent performance, with estimated throw distances closely matching the true measured distance. The results indicate that the SmartBall system can produce reliable sports analytical data using only onboard sensing, a calibrated motion model set for a specific environment, and the base station.

3.1 Experimentation

To evaluate the SmartBall system, we performed a series of controlled experiments in a measured indoor environment. The experimental setup consisted of repeatedly throwing the SmartBall between two fixed points that are approximately 11 ft (3.35m) apart. The SmartBall was internally configured for this environment to increase the accuracy of the performance metrics. The throw origin and target location were kept as consistent as possible throughout testing to try and minimize environmental variability and ensure repeatability. During each trial, the SmartBall's embedded hardware monitored motion data continuously to

detect a throw event. When a throw event occurred, the system transmitted motion data samples, including both pre-throw and post-throw information to the base station via UDP over Wi-Fi.

The Python listener running on the base station processed incoming data at a sampling rate of 200Hz (equates to one sample per 5ms). For each detected throw, the system computed several metrics including:

- Release Time (s)
- Release Speed (m/s)
- Peak Acceleration (m/s^2)
- Peak Rotation (rad/s)
- Expected Distance (m)

The listener also applied filtering and validation checking to ensure data quality, rejecting data points that fell outside of acceptable parameters. The throws that were rejected are marked invalid and excluded from analysis. Examples of rejected data points are insufficient sample count, poor pre-throw stability, and unrealistic computed values. A total of 38 valid throws were collected and used for experimental performance analysis.

3.2 Experiment Results

The experimental results demonstrate that the SmartBall system is capable of consistently extracting meaning motion data and accurately estimating throw distance in a controlled environment. A total of 38 valid throws were analyzed, with summary statistics shown in Figure 3.

Average Analytical Data	
Release Time (s)	0.749545455
Release Speed (m/s)	7.940818182
Peak Acceleration (m/s^2)	183.9351364
Peak Rotation (rad/s)	26.13995455
Expected Distance (m)	3.480409091
Total Throws	38

Figure 3: Average analytical metrics computed across 38 valid throws.

Across all trials, the average release time was ~ 0.75 seconds which indicates a consistent duration of force being applied to the ball prior to release. The system measured an average release of 7.94 m/s which is within a realistic range for the short distance throws performed in our controlled environment. This consistency in throw release timing and speed suggests that our experimental conditions were both stable and repeatable. The SmartBall also captured high-intensity motion data as there was an average peak acceleration of 183.94 m/s^2 and an average peak rotational velocity of 26.14 rad/s. These values reflect the rapid force application and spin that is imparted upon the ball during a typical throw. This demonstrates the SmartBall system's ability to accurately measure both linear and rotational motion using the MPU6050 within the ball. Our most important result is that the system produced an average estimated throw distance of 3.48m which is close to the true experimental distance of 3.35m (11ft). This corresponds to an average error of $\sim 4\%$ which shows

that the calibrated velocity-based model used by the listener provides accurate and reliable distance predictions within a controlled environment.

The reason for our ~4% average error rate can be due to various factors. First, the experiment was performed repeatedly in a controlled environment which minimizes variability between each throw. Next, the listener's data validation and filtering only includes throws that meet a realistic standard. Lastly and most importantly, the system was calibrated for the controlled environment the experiment took place within which improves the accuracy of the mapping between velocity data and estimated distance. While we believe these results are promising, the performance of the system is heavily dependent on optimization for a controlled environment. Variations in throw angle and speed or factors like air resistance, all of which appears in more realistic use-cases, can introduce more error into the results. Overall, the results show that the SmartBall system can reliably capture sports performance metrics and estimate throw distance with good accuracy, supporting its feasibility as a low-cost, portable alternative to traditional systems.

4. Limitations

There are several constraints in the system which may hinder its effectiveness in real-world use. The validation tests were completed in an indoor setting, with consistent distances for each throw; therefore, in an outdoor environment changes in angles, surfaces, and other factors, may result in a decreased accuracy. The IMU sensor system is prone to error and drift, which may cause inaccuracy in the measurement of velocity, angle displacement, and distance. Distance calculations through models of projectiles disregard the effects of air resistance and unpredictable throw movements. Motion detection through the UDP listening algorithm may wrongly consider unnecessary movements as throws. Acceleration measurements are limited by sample rates and resolutions, making it hard to detect sudden acceleration. Furthermore, acceleration measurements depend on the release angle, which must be calibrated. Moreover, the restrictions for the capability of onboard processing make the sophistication of the filtering and modeling methods more limited than those possible with offboard processing. The battery life dictates both how long the device is able to operate without having to be recharged and the sampling ability of the device. The wireless communication poses a problem in terms of error due to packet losses or latencies. Incorporating the electronics into the ball makes a difference in the mass distribution compared to that of a normal ball.

5. Future Work

Future work of the SmartBall system will focus on implementing improvements to improve the accuracy of throw data detection and calculation, in addition to better applicability in less stable conditions, such as outdoor play, which is a common setting for playing sports. Additionally, a magnetometer would be implemented into the embedded circuit to enable global context within the system, thus enabling calculation of predicted distance and trajectories. While the current implementation is bulky due to each component being on a separate circuit, a custom PCB may be used to integrate all components onto one board to reduce the footprint of the system. Onboard charging would further reduce limitations on the system, and make the system more practical by removing the need for disassembly of the ball for charging. In terms of improving the communication robustness, we can utilize the capability of the ESP32 to operate in AP mode in areas with no Wi-Fi. The long-range mode of the ESP32 may also be utilized to extend ball communication up to 1km, which is more than sufficient for real-world use in sport analytics.

6. Conclusion

In conclusion, the SmartBall system is a low-cost, and portable alternative to traditional sports analytics systems. The system provides real-time throw detection and wireless data transmission, allowing for fast extraction of key metrics to enable calculation through a throw processing pipeline. The SmartBall was shown to have a ~4% distance estimation error in a controlled environment, thus proving the feasibility of low-cost embedded hardware use in sports analytics to capture and analyze motion in real time. Although full trajectory estimation is limited due to a current lack of hardware, the system provides a strong foundation for future improvements, such as global tracking and miniaturization, to make the system more robust and practical for real-world use.

References

Mănescu, D. C. (2025). Big Data Analytics framework for decision-making in sports performance optimization. *Data*, 10(7), 116. <https://doi.org/10.3390/data10070116>